Examination 2024-10-31

Introduction to Parallel Programming (5 hours)

Please make sure to write your exam code on each page you hand in. When you are finished, please put the pages containing your solutions together in an order that corresponds to the order of the questions and mark the questions you have answered in the exam front sheet. You can take with you this exam if you want. This is a **closed book** examination but you are allowed to have with you an **A4 sheet of prepared hand-written notes.** It contains **40** points in total and their distribution between sub-questions is clearly identifiable. To get a final grade of **5** you must score at least **34**. To get a grade of **4** you must score at least **28** and to pass the exam you must score at least **19**. Note that you will get **credit only for answers that are correct (or a very serious attempt) and clear.** All answers should preferably be **in English**; if you are uncomfortable with English you might of course use Swedish, but note that for most of the questions only relatively short answers using natural language are required. Whenever in *real doubt* for what a particular question might mean, **state your assumptions clearly.**

DON'T PANIC!

1. **General Questions ($3 \times 2 = 6$ pts total).** Give *brief* answers to the following questions:
   (a) What are condition variables and how are they used in multi-threaded programming?
   (b) What do the terms blocking and non-blocking synchronization mean? How do these two synchronization mechanisms compare to each other?
   (c) What are examples of parallel programs that OpenMP *tasks* allow us to realize, perhaps more easily, compared to OpenMP versions (prior to 3.0) that do not support tasks?

---

2. **Amdahl's Law ($2 \times 2 = 4$ pts total).** Use Amdahl's Law to resolve the following questions:
   (a) Suppose a computer program has a method M that cannot be parallelized, and that this method accounts for 30% of the program's execution time. What is the limit for the overall speedup that can be achieved by running the program on an n-processor multiprocessor machine?
   (b) Suppose the method M can be sped up three-fold. What fraction of the overall execution time must M account for in order to double the overall speedup of the program?

---

3. **Efficiency of Parallelism ($2 + 1 + 1 = 4$ pts total).** Suppose that $T_{serial} = n$ and $T_{parallel} = n/p + log_2(p)$, where p is the number of processes. If we increase p by a factor of k, find a formula for how much we will need to increase n in order to maintain constant efficiency E (recall $E = S/p$, where S is the parallel speedup). How much should we increase n by if we double the number of processes from 4 to 8? Is the parallel program scalable?

4. **Data Races ($3 \times 2 = 6$ pts total).** Which of the following programs are data race free (i.e., they do not contain a data race)? Justify your answer by either showing a 'racy' execution or by giving a reason why there cannot be a data race.

   (a) Thread 1: `lock m; *x = 1; unlock m; *y = 1`
   Thread 2: `lock m; r = *x; unlock m; if (r = 1) then print *y`

   (b) Thread 1: `*y = 1; lock m; *x = 1; unlock m;`
   Thread 2: `lock m; r = *x; unlock m; if (r = 1) then print *y`

   (c) Thread 1: `*y = 1; lock m; *x = 1; unlock m;`
   Thread 2: `lock m; r = *x; unlock m; if (*x = 1) then print *y`

   where `m` is a monitor (lock), `x` and `y` shared-memory locations and `r` is a thread-local variable. Assume that all memory locations are zero-initialised.

---

5. **Locks ($3 + 2 = 5$ pts total).** The following code implements a *Ticket Lock*. To access a critical section, we take a ticket, and wait until it is our turn. After we are done in the critical section, we state that the next one in line is allowed to enter by incrementing `current`.

```
class TicketLock {
  int next = 0;
  int current = 0;

  void lock() {
    int me = next;
    next += 1;
    while(current != me) {}
  }

  void unlock() { current++; }
}
```

   (a) Is this code deadlock free? Does it guarantee mutual exclusion? Provide the reasoning behind your answers.

   (b) There is at least one issue with the implementation. What is it, and how would you fix it with *minimal* modifications?

---

6. **OpenMP (4 pts).** Complete the following program that, given an array of integers `A[N]`, uses OpenMP to count the number of elements of this array that are zero. Your task is to write the ... code and explain the OpenMP constructs you have chosen to use and why:

```
#define N 12345678
int main(int argc, char *argv[]) {
  int A[N], i, totalZero = 0;
  // Here we initialize A with values

  ...  // You need to write this code

  // Now we have the answer
  printf("Number of zeros is %d\n", totalZero);
}
```

7. **Queue Locks ($2 \times 3 = 6$ pts total).** The code shown below is an alternative implementation of CLHLock in which a thread reuses its own node instead of its predecessor node.

```
public class BadCLHLock implements Lock {
  // most recent lock holder
  AtomicReference<Qnode> tail = new AtomicReference<QNode>(new QNode());
  // thread-local variable
  ThreadLocal<Qnode> myNode = new ThreadLocal<QNode> {
    protected QNode initialValue() {
      return new QNode();
    }
  };
  public void lock () {
    Qnode qnode = myNode.get();
    qnode.locked = true; // I'm not done
    // Make me the new tail, and find my predecessor
    Qnode pred = tail.getAndSet(qnode);
    // Spin while the predecessor holds lock
    while (pred.locked) {}
  }
  public void unlock() {
    // reuse my node next time
    myNode.get().locked = false;
  }
  static class Qnode { // Queue node inner class
    public boolean locked = false;
  }
}
```

(a) Carefully explain how this implementation can go wrong.

(b) Explain how the MCS lock avoids the problem even though it reuses thread-local nodes.

---

8. **MPI (5 pts).** Consider the following program that calculates π based on the number of random points within a unit circle. Parallelize this program by adding pseudo-MPI code.

```
double x, y, dist, pi;
int i, incircle = 0;

for (i = 0; i < N; i++) {
  x = (double)rand() / RAND_MAX;
  y = (double)rand() / RAND_MAX;

  dist = x * x + y * y;

  if (dist <= 1) {
    ++incircle;
  }
}
pi = 4.0 * incircle / N;
```

Good luck !